

Programming

Damir Krstinić
damir.krstinic@fesb.hr

About...

- Computing
- Computation
- Computers
- Computer science

Algorithm

- Precisely specified procedure that defines required steps and their order of execution to accomplish given task

FINITE: the number of steps in the computation is finite!

DEFINITE: precisely specified set of a input and output values

UNAMBIGUOUS: distinct set and order of operations (given in the language understandable by the tool!)

COMPLEXITY: measure of the requirements for the algorithm execution (memory, time, ...)

Program

- ♦ Describes steps of the algorithm in the manner understandable to the computer
- ♦ Description of the algorithm in some computer language
- ♦ Programming requires familiarity with syntax, semantics and “internal logic” (paradigm) of the programming language
- ♦ Data structures are integral part of the program
- ♦ Input data, internal data during execution of the program, output data
- ♦ Data model determine properties of the algorithm and the result in computer program

Programming

- ♦ Unambiguously defining steps of execution to be taken to solve the problem

```
Read first number from keyboard;  
Read second number from keyboard;  
Compute the sum of two numbers;  
Write result on screen;
```

- ♦ Algorithm described in some computer language together with the data structures constitute a computer program

Machine code

- ♦ All functions of digital computers are realized with a limited set of simple primitives
 - ♦ Set of primitive operations that can be directly executed on the processor (CPU – Central processing Unit) are called machine code
- ♦ Sequence of machine code instructions constitute an executable program
- ♦ Every program written in higher level languages must be translated to a machine code to be executed

Machine code

- Machine code commands are encoded as numbers
- Each machine code instruction has its symbolic name (assembly language, or assembler)
- Sequence of machine code instructions constitute an executable program
- Writing programs is extremely complicated and requires profound knowledge of the architecture of the processor and deep understanding of the principles of computer operation

Higher level languages

- ♦ Closer to the human language (and way of thinking)
 - ♦ Commands are usually based on the natural language (English)

```
a = input('Enter first number: ');  
b = input('Enter second number: ');  
c = a + b;  
disp(c);
```

- ♦ Computer programmer can think about the algorithm and the program goal, instead of the details of the technical implementation of the computer

Program: Add two numbers

```
a = input('Enter first number: ');  
b = input('Enter second number: ');  
c = a + b;  
disp(c);
```

Program: Add two numbers

command: **read data from the keyboard**

```
a = input('Enter first number: ');  
b = input('Enter second number: ');  
c = a + b;  
disp(c);
```

Program: Add two numbers

command: **read data from the keyboard**

Argument of the command (text)

```
a = input('Enter first number: ');  
b = input('Enter second number: ');  
c = a + b;  
disp(c);
```

Program: Add two numbers

operator: =

variable (symbolic
name **a**)

command: **read data from the keyboard**

Argument of the command (text)

```
a = input('Enter first number: ');  
b = input('Enter second number: ');  
c = a + b;  
disp(c);
```

Program: Add two numbers

operator: =

```
a = input('Enter first number: ');  
b = input('Enter second number: ');  
c = a + b;  
disp(c);
```

operator: +

variable (symbolic name **c**)

Program: Add two numbers

```
a = input('Enter first number: ');  
b = input('Enter second number: ');  
c = a + b;  
disp(c);
```

argument of the command (variable **c**)

command: **write data on screen**

Program: Add two numbers

operator: =

variable (symbolic
name **a**)

command: **read data from the keyboard**

Argument of the command (text)

```
a = input('Enter first number: ');  
b = input('Enter second number: ');  
c = a + b;  
disp(c);
```

operator: +

argument of the command (variable **c**)

command: **write data on screen**

From algorithm to program

```
Read first number from keyboard;  
Read second number from keyboard;  
Compute the sum of two numbers;  
Write result on screen;
```

algorithm

From algorithm to program

```
Read first number from keyboard;  
Read second number from keyboard;  
Compute the sum of two numbers;  
Write result on screen;
```

Coding in the
computer
language

algorithm



program

```
a = input('Enter first number: ');  
b = input('Enter second number: ');  
c = a + b;  
disp(c);
```

Translation of program to machine code

- Regardless of a computer language used, all programs have to be converted to a machine code to be executed on a computer.

Machine code does not know complex operations like:

Read number from keyboard

- Each command of a higher level language is translated to a sequence (often very complex) of primitives (machine code instructions)

Translation of program to machine code

Two basic techniques:

INTERPRETING:

- ◆ Each command is translated separately and immediately executed

COMPILING:

- ◆ Program written in higher level language is converted to a machine code as a whole, generating single executable file (commands cannot be executed separately)

Interpreters

- ◆ Commands of the source code are translated one at a time, generated machine code is executed right after translation of the command
- ◆ Sequence of commands (a program) can be stored in a text file. This text file can be started as an executable program, but interpreter still reads one command at a time from the file and translates each command separately as it was entered by a user one after another

Interpreters

- ♦ Advantage: simplicity
 - ♦ Interpreter allows interactive programming.
Outcome of each command is visible right after the command is issued, easy to find errors
- ♦ Deficiency: interpreters are slow
 - ♦ Translation process is started for each command, each time a program is started (machine code executable is never generated)
 - ♦ Interpreter must be installed on the computer on which we intend to start the program

Compilers

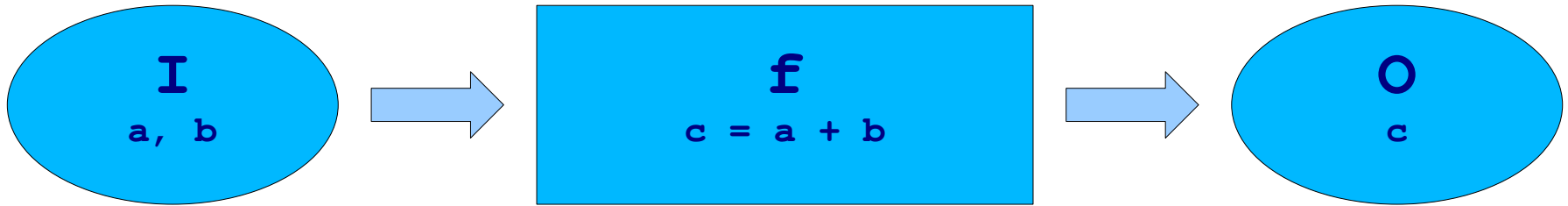
- ▶ Program in source code is translated to the machine code executable as a whole
- ▶ Compiling process is started once. Compiler reads whole program written in higher level language and generates the executable file (sequence of machine code instructions)
- ▶ Generated executable file that can be directly executed

Compilers

- ♦ Advantage: efficiency
 - ♦ Execution of the executable file as very fast, no additional steps are needed
 - ♦ When program is translated as the whole, further optimizations are possible
- ♦ Deficiency: complexity
 - ♦ Once translated to the executable, program can not be altered
 - ♦ Modifications are possible in the source code

Algorithm and data model

- Every algorithm can be observed as a function f mapping input data I to output data O



- ALGORITHM:** describes function f
- DATA MODEL:** defines internal representation of the data and access mode

Data model

- ◆ Defines data to be used, including input data, output data and internal data
 - ◆ declares symple (primitive) data (integer, real, textual, ...), symbolic variables names, dimensionality of the data, restrictions
- ◆ Defines abstract data types
 - ◆ Data structures (data organized in logical units)
 - ◆ Allowed operations on abstract data

Primitive data types

- ♦ numerical
 - ♦ **integer**
 - ♦ Single precision floating point numbers (**float**), double precision floating point numbers (**double**)
- ♦ character data
 - ♦ Single character (**character**), or sequence of characters (**string**)
- ♦ logic data (**boolean**)
 - ♦ can take one of two states: **TRUE** or **FALSE**

Primitive data types

- ◆ Data type defines:
 - ◆ Allowed values that given variable can take
 - ◆ Set of allowed operations
 - ◆ Model for storing data in memory
- ◆ Data of the same type can be compared (for example to find bigger or smaller value)
- ◆ Sequence of data of the same type can be stored in the array or vector

Vector

- ◆ Simplest of all data structures
 - ◆ Allows direct access to every element of the vector based on its index

```
a[1] = input('Enter first number: ');  
a[2] = input('Enter second number: ');  
a[3] = a[1] + a[2];  
disp(a[3]);
```

- ◆ Adjacent elements of the vector are stored on successive memory locations

Array

- ♦ As opposed to a vector, array has a fixed size
 - ♦ Number of elements must be assigned in advance

```
float a[3];  
a[1] = input('Enter first number: ');  
a[2] = input('Enter second number: ');  
a[3] = a[1] + a[2];  
disp(a[3]);
```

NOTE:

In some programming languages indexes of array elements start with 0 instead of 1!

Multidimensional arrays

- Allow organization of the data in several dimensions

```
A = rand(3,2);  
B = [1 3 2 1; 2 3 1 4];  
C = A*B;  
size(C)  
(program output:)  
ans = 3 4  
C(1,3)  
(program output:)  
ans = 2.4813
```

Element of a multidimensional array is accessed by multidimensional index (as in the above example)

Apstraktni tipovi podataka

- ▶ Apstraktni tip podataka (ATP) definiran je modelom podataka te dozvoljenim operacijama nad podacima.
 - ▶ Dobra organizacija podataka olakšava razvoj algoritma, smanjuje mogućnost i olakšava pronalaženje greške, olakšava pohranu i prezentaciju podataka korisniku
 - ▶
- ▶ ATP ne definira konkretnu prezentaciju podataka u memoriji niti način izvršavanja operacija

Apstraktni tipovi podataka

- ▶ Apstraktni tip podataka (ATP) definiran je modelom podataka te dozvoljenim operacijama nad podacima.
 - ▶ Dobra organizacija podataka olakšava razvoj algoritma, smanjuje mogućnost i olakšava pronalaženje greške, olakšava pohranu i prezentaciju podataka korisniku
 - ▶
- ▶ ATP ne definira konkretnu prezentaciju podataka u memoriji niti način izvršavanja operacija

Struktura

- Struktura je skup različitih podataka u jedinstvenoj cjelini

```
s = struct('ime', 'Damir', 'ocjena', 2);  
s.ime
```

(program output:)

```
ans = Damir
```

```
s.ocjena
```

(program output:)

```
ans = 2
```

```
s.ocjena = 5;
```

Članovima strukture pristupamo preko imena varijable i imena člana strukture

“Logična” organizacija podataka: podaci koji pripadaju zajedno čine cjelinu

Lista

- ♦ Apstrakcija niza članova s definiranim poretком

```
s1 = struct('ime', {'Ivan','Damir'}, 'ocjena', {2,5});  
s1(3) = struct('ime', 'Branko', 'ocjena', 3);  
size(s1)
```

```
(program output:)  
ans = 1 3
```

```
---
```

```
s1(2)
```

```
(program output:)
```

```
ans =  
{  
    ime = Damir  
    ocjena = 5  
}
```

Najjednostavnija implementacija liste je vektor ili polje

Podacima pristupamo preko imena varijable (simboličko ime za listu) i indeksa elementa

U ovom primjeru, član liste je struktura

Vezana lista

- ♦ Lista u kojoj članovi liste imaju međusobne reference
 - ♦ Najjednostavniji oblik je jednostruko vezana lista u kojoj svaki član liste pokazuje na slijedeći:

```
s2 = struct ('ime', 'Damir', 'ocjena', 5, 'next', '');  
s2.next = struct ('ime', 'Ivan', 'ocjena', 2, 'next', '');  
s2.next
```

(program output:)

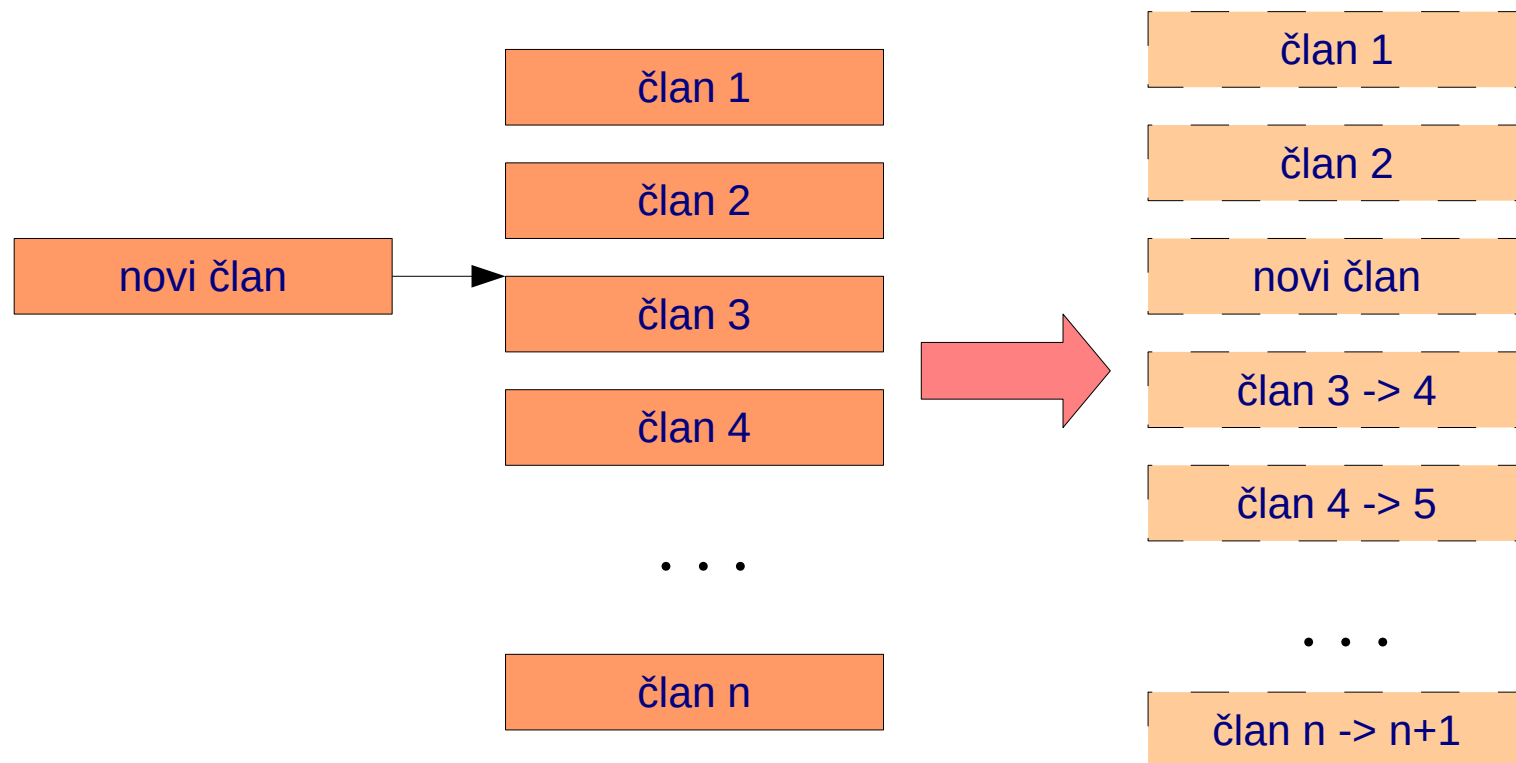
```
ans =  
{  
    ime = Ivan  
    next =  
    ocjena = 2  
}
```

Veća sloboda manipulacije članovima liste
(npr. lakše umetanje)

Sporiji pristup članovima

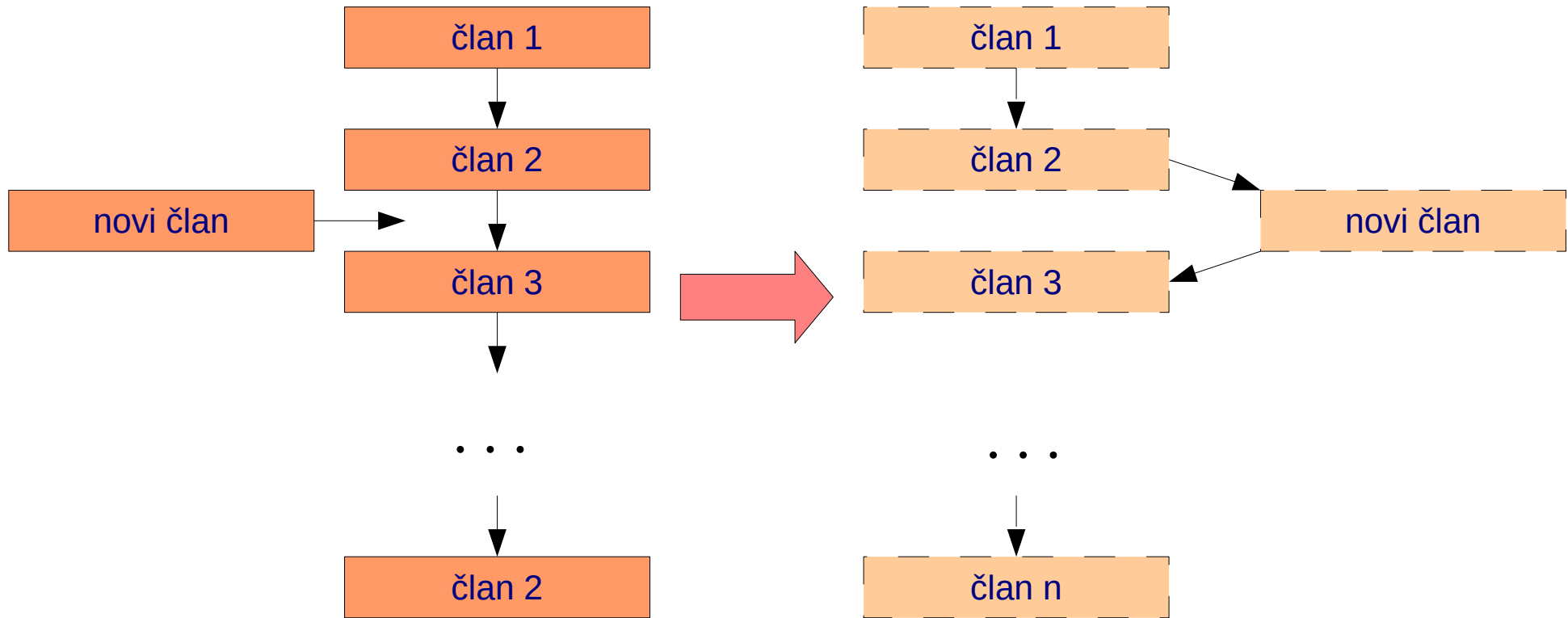
Za pravu implementaciju vezane liste
potrebno je bolje poznavanje programiranja

Umetanje člana u listu



Kod umetanja člana u listu, svi članovi koji se nalaze iza mjesta umetanja moraju se pomaknuti za jedno mjesto naprijed

Umetanje člana u vezanu listu



Kod umetanja u vezanu listu, dovoljno je preusmjeriti referencu člana iza kojeg se novi član dodaje, i referencu novog člana postaviti na sljedeći

Stog (stack)

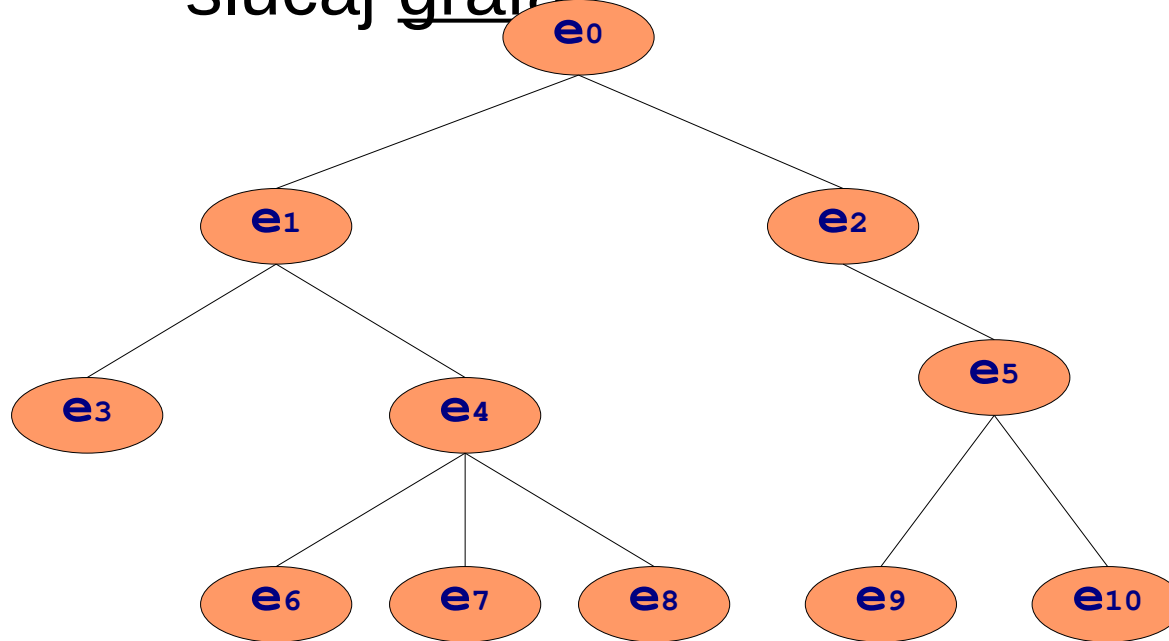
- ♦ Struktura podataka sa LIFO (Last In, First Out) pristupom.
 - ♦ Član se može postaviti na vrh stoga (**push**), ili skinuti sa vrha stoga (**pop**)
- ♦ Implementacija:
 - ♦ Vektor (lista): elementi vektora + pokazivač (indeks člana) na vrh stoga
 - ♦ Vezana lista: reference prema “dnu” stoga. Novi član dodaje se na način da se postavi referenca na član koji je prethodno bio na vrhu stoga.

Red (Queue)

- ♦ Struktura podataka sa FIFO (First In, First Out) pristupom
 - ♦ Novi član dodaje se na kraj reda, preuzima se sa početka
- ♦ Implementacija:
 - ♦ Vektor: članovi + pokazivači na početak i kraj reda (ovi pokazivači postati neograničeno veliki, a da je pri tom u redu uvijek samo jedan član!)
 - ♦ Cirkularni niz: Kapacitet se povećava po potrebi
 - ♦ Vezana lista: član dodajemo na kraj liste, uzimamo sa početka

Stablo

- Stablo je rekurzivan struktura u kojoj svaki član ima **0** ili više podstabala (matematički, stablo je poseban slučaj grafa)



Svaki čvor ima *0 ili više* djece
Svaki čvor ima točno jednog roditelja

Korijen je čvor bez roditelja

List je čvor bez djece

Čvorovi sa istim roditeljem su siblinzi

Stupanj stabla je najveći broj djece koji posjeduje neki čvor u stablu

Visina stabla je duljina najduljeg puta od korjena prema listu

Programerske paradigme

- Sekvencijalni jezici

- Cijeli kod programa u jednom bloku, naredbe se izvode redom kojim su navedene

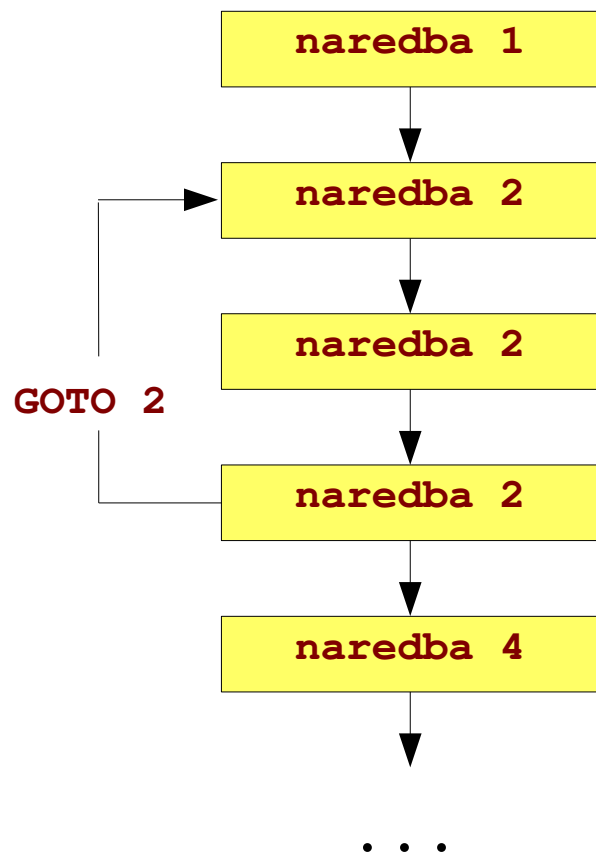
- Proceduralni jezici

- Kod je organiziran u logičke cjeline koje nazivamo funkcije ili procedure. Svaku proceduru moguće je pozvati više puta u raznim djelovima koda (čak i iz druge procedure)

- Objektno orijentirani jezici

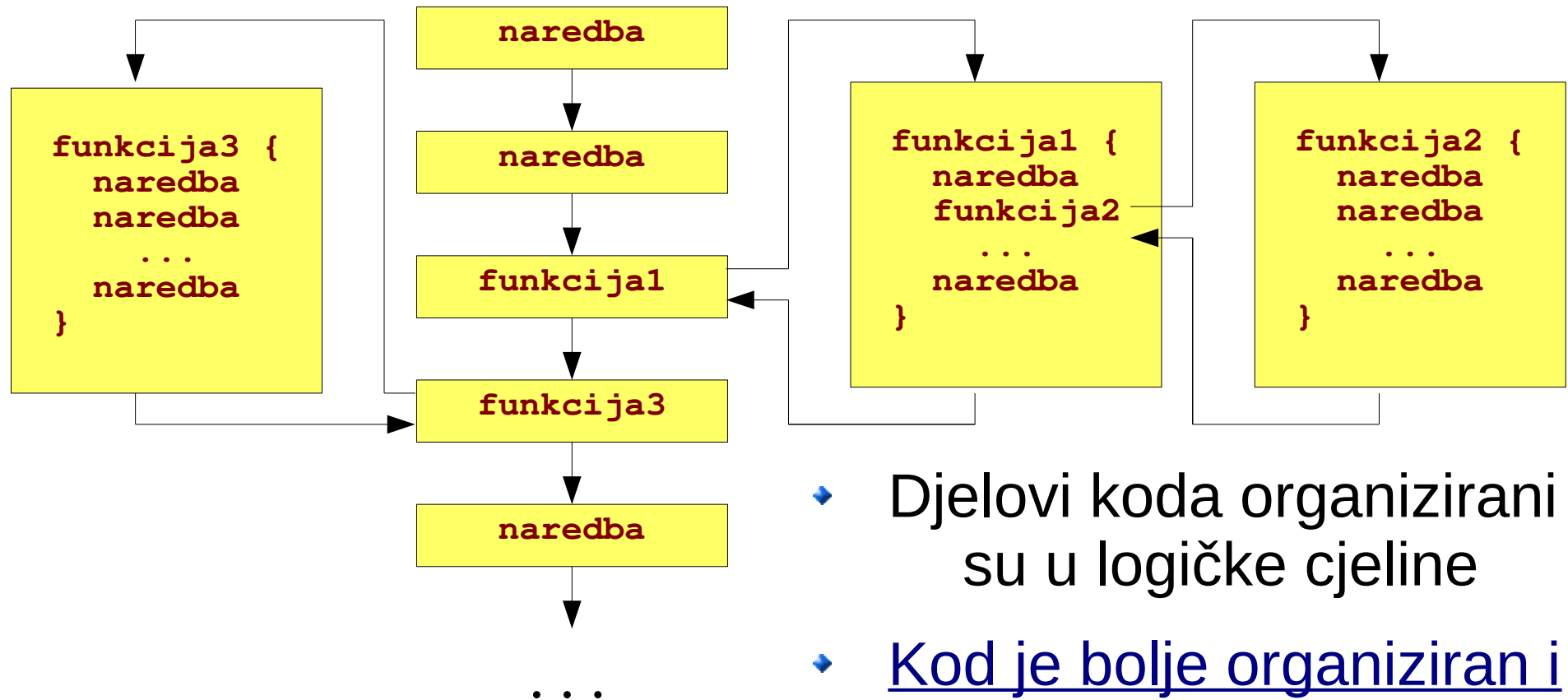
- Program čini skup objekata koji objedinjuju podatke i funkcije za pristup podacima. Međusobnom interakcijom objekata realizira se program.

Sekvencijalni jezici (BASIC)



- ▶ Naredbe se izvršavaju redom kojim su navedene u programu
 - ▶ Naredbe su u pravilu numerirane
- ▶ Eventualno grananje realizira se skokovima (naredba **GOTO**) unutar koda
- ▶ Kod nije logički organiziran, teško se čita, pogotovo ako su česta grananja

Proceduralni jezici (C, Pascal)



- Djelovi koda organizirani su u logičke cjeline
- Kod je bolje organiziran i pregledniji

Objektno orjentirano programiranje

- ♦ Programerska paradigma prema kojoj program čini skup individualnih jedinica (objekata) sa mogućnošću interakcije
 - ♦ Klasični pristup programiranju: program čini kolekcija funkcija ili jednostavno lista naredbi
- ♦ Svaki objekt je nezavisna jedinica sposobna primiti i slati poruke, te obrađivati vlastite podatke
 - ♦ Objekti istog tipa čine klasu (**Class**) objekata.

Klase i objekti

- ♦ Klasa definira apstraktne karakteristike objekta
 - ♦ Apstraktna struktura sa definiranim podacima i operacijama za pristup i manipuliranje podacima
- ♦ Objekt je instanca klase
 - ♦ Klasa **Automobil** definira sve automobile, objekt **moj_auto** je jedan određen automobil

```
Class Automobil {  
    sučelje {  
        volan();  
        gas();  
        kočnica();  
    }  
    privatne_metode {  
        trakcija();  
    }  
    podaci {  
        brzina, akceleracija;  
        težina, gume;  
        količina_goriva;  
    }  
}
```

Interakcija među objektima

- ♦ Klasa Vozač prima poruke kroz sučelje:
 - ♦ **vid(), volan()**
 - ♦ Mijenjaju se privatni podaci klase
- ♦ **odlučivanje()**
 - ♦ Pozivaju se metode sučelja klase **Automobil**

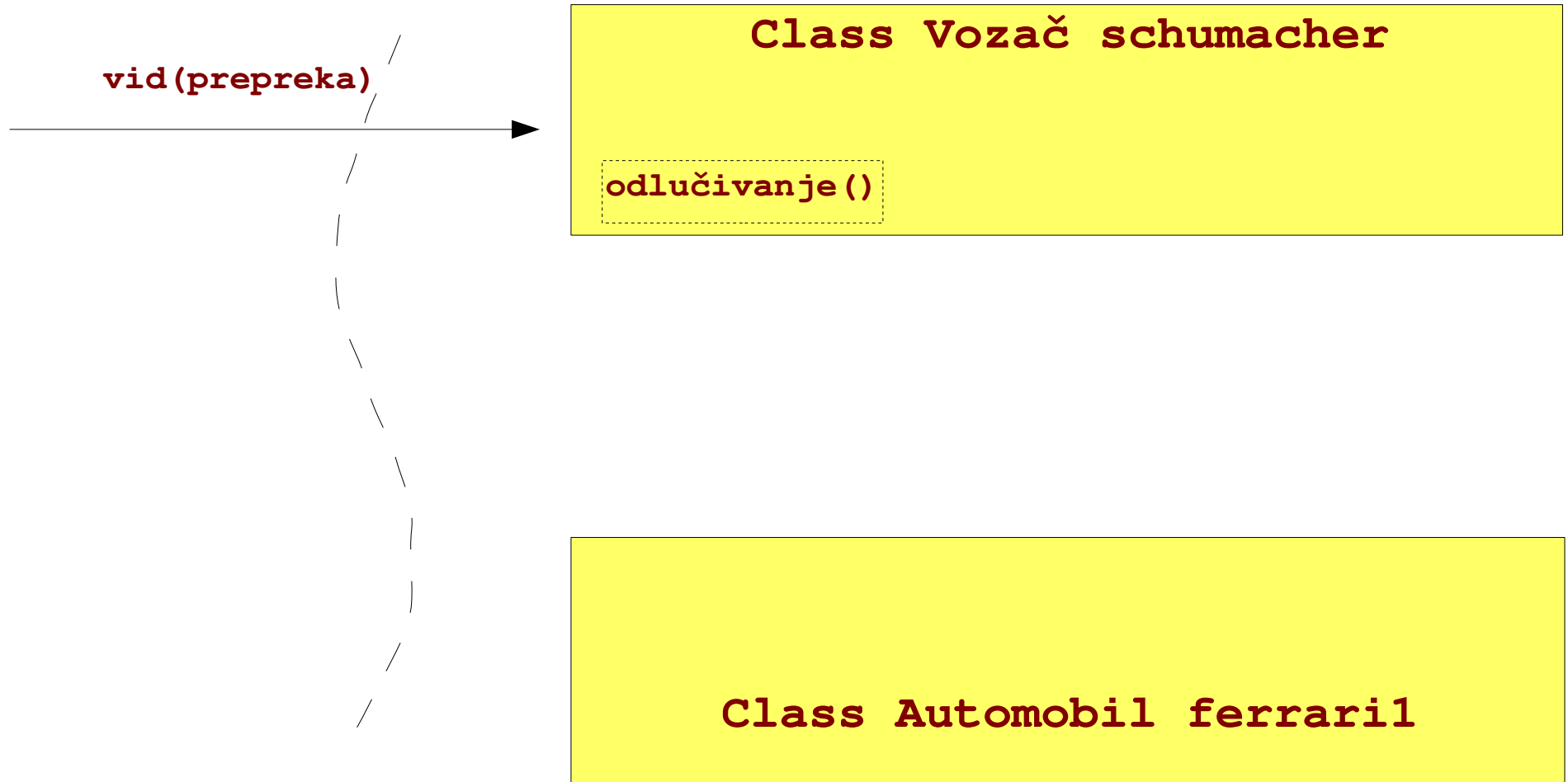
```
Class Vozač {  
    sučelje {  
        vid();  
        volan();  
    }  
    privatne_metode {  
        odlučivanje();  
    }  
    podaci {  
        vektor ostali_automobili;  
        struktura stanje_cesta;  
        brzina, umor;  
    }  
}
```

Program Formula1

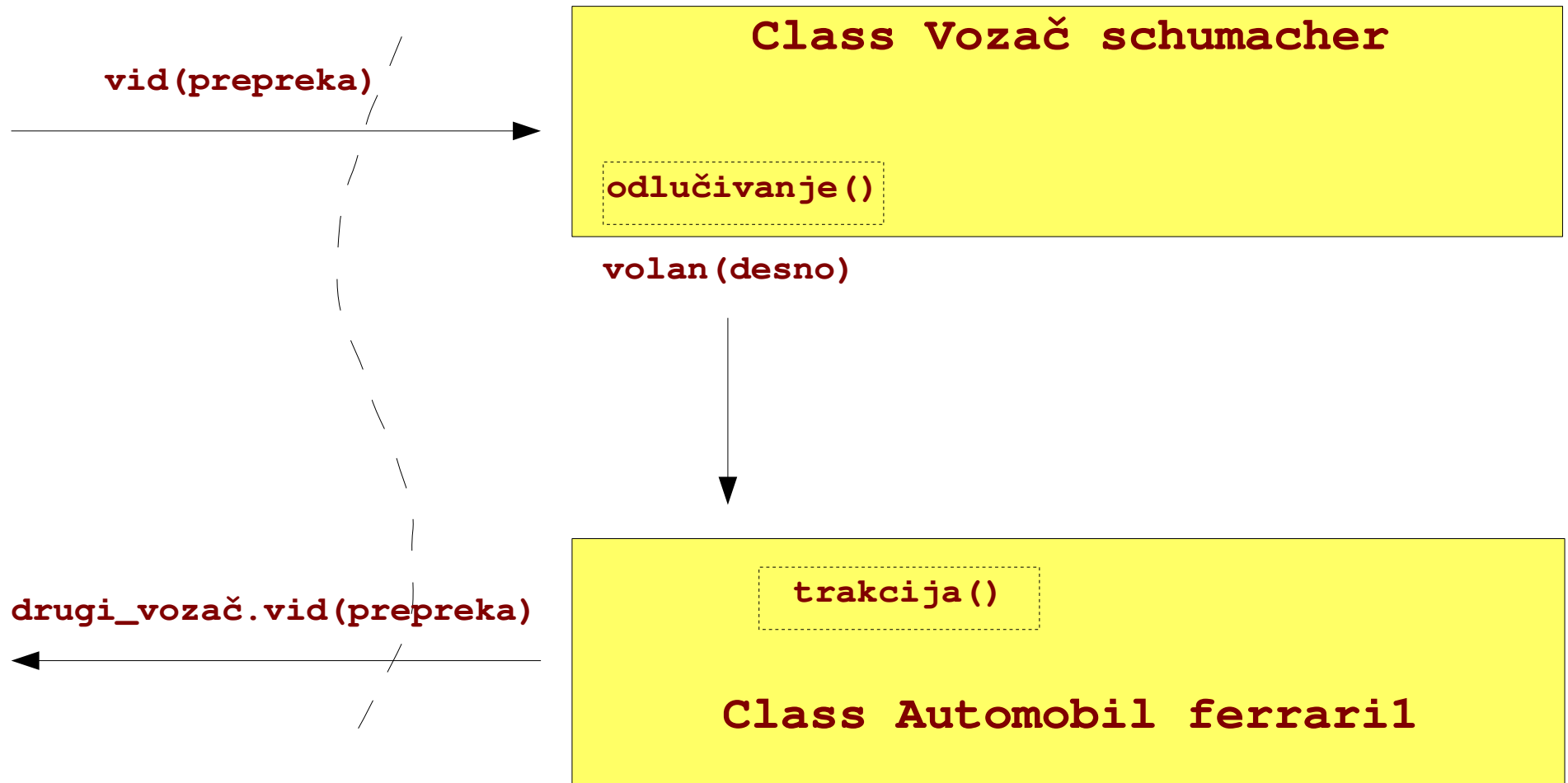
Class Vozač schumacher

Class Automobil ferrari1

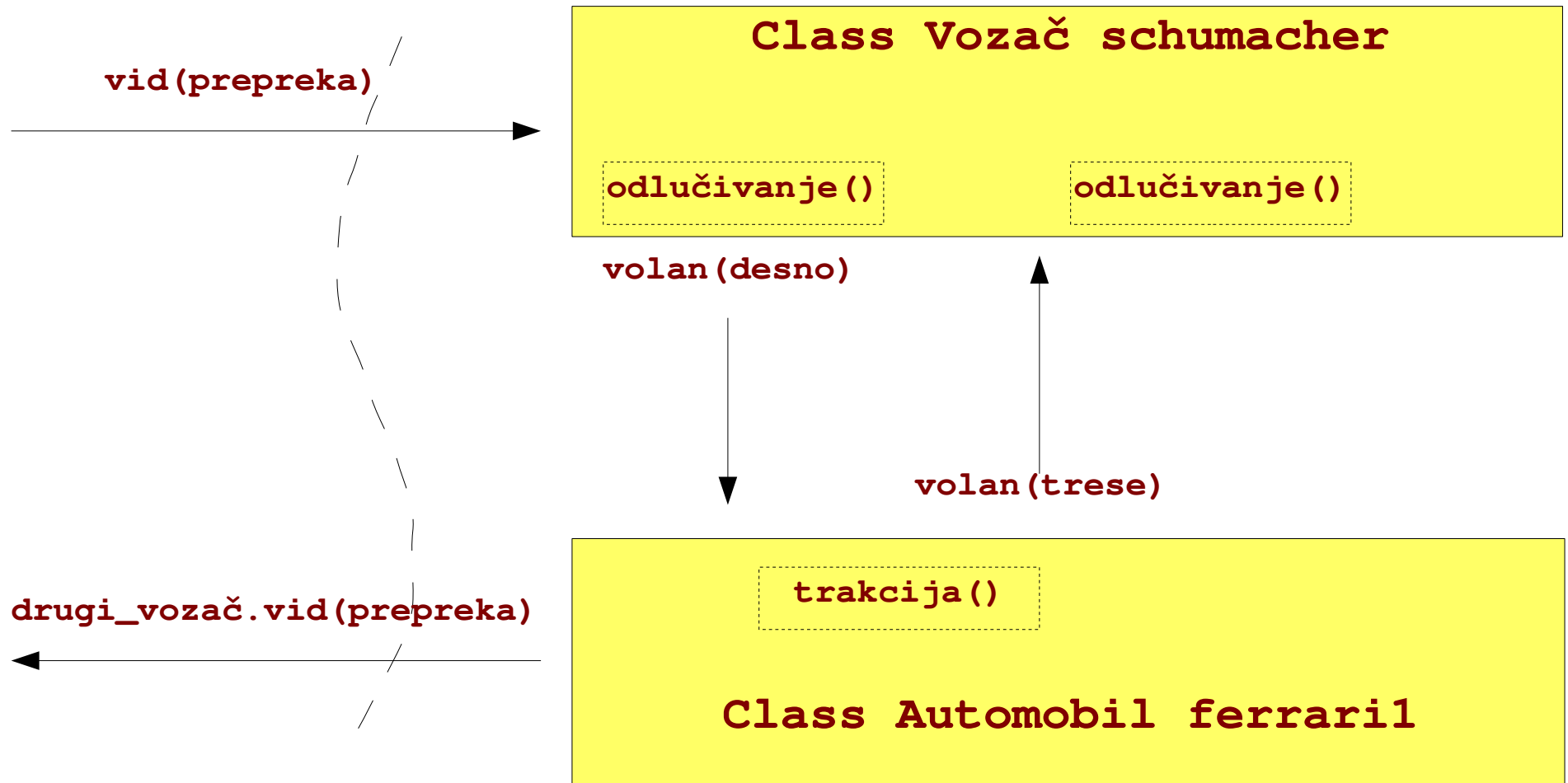
Program Formula1



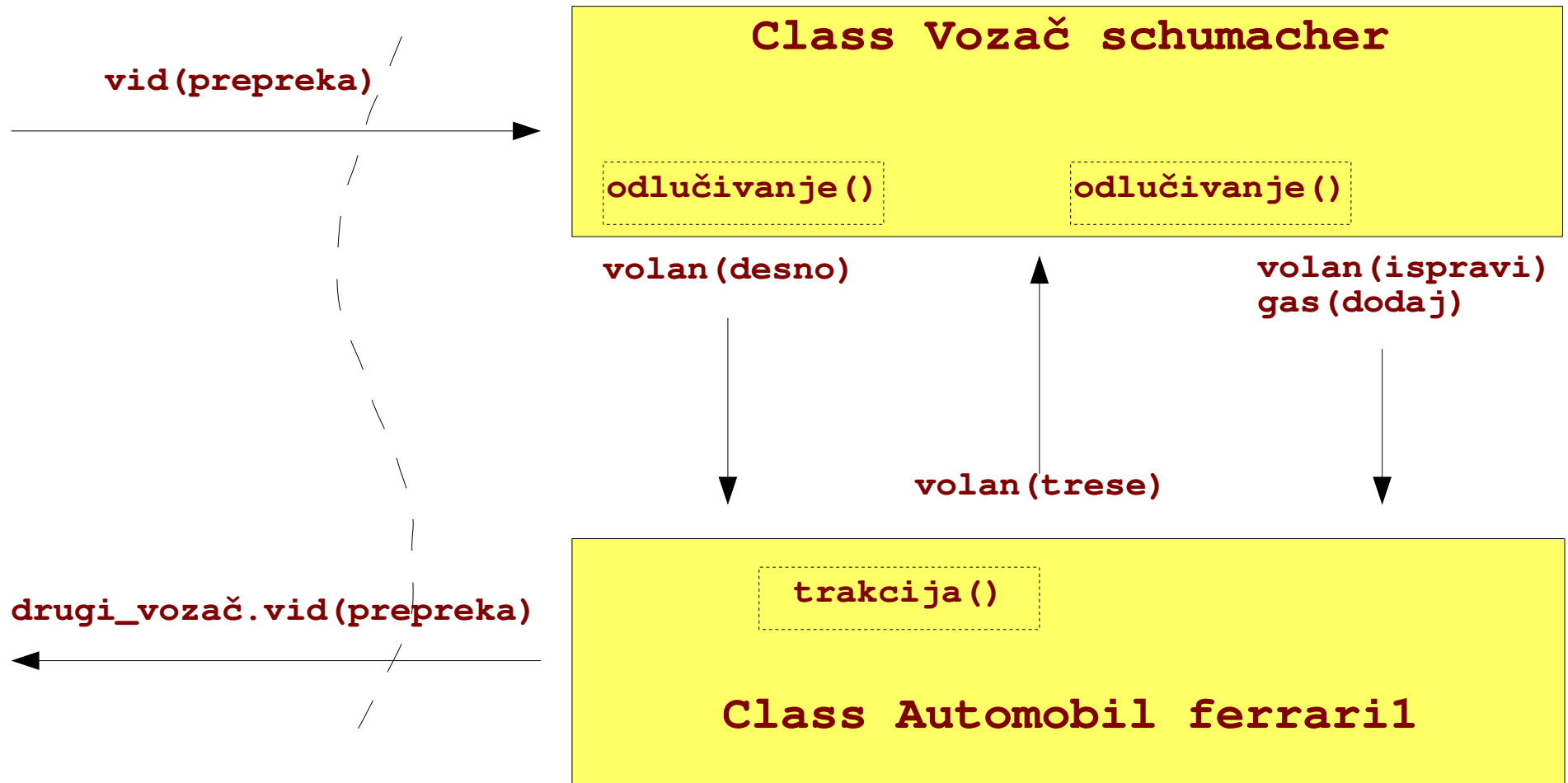
Program Formula1



Program Formula1



Program Formula1



What is a Computer science?